

ACRS: Adjacent Computation Resource Sharing among Partitioned GPU Sub-Cores

Penghao Song^{1,2}, Chongxi Wang^{1,2}, Chenji Han^{1,2}, Haoyu Zhao³, Tingting Zhang³, Tianyi Liu^{4,*}, Jian Wang^{1,2,*}

¹State Key Lab of Processors, Institute of Computing Technology, CAS, Beijing, China

²University of Chinese Academy of Sciences, Beijing, China

³Loongson Technology Co. Ltd, Beijing, China

⁴University of Texas at San Antonio, United States

{songpenghao16,wangzhongxi15,hanchenji16}@mailsucas.ac.cn,

{zhaohaoyu,zhangtingting}@loongson.cn, tianyi.liu@utsa.edu, jw@ict.ac.cn

Abstract—Modern GPUs typically segment Streaming Multiprocessors (SMs) into sub-cores (e.g. 4 sub-cores) to reduce power consumption and chip area. However, this partitioned design prevents potential task distributions across sub-cores, impairing overall execution efficiency. In this paper, we explore the performance benefit of sharing hardware resources among sub-cores and identify functional units (FUs) as critical components for compute-intensive applications. Moreover, our observations reveal that instructions residing in operand collectors can be obstructed by back-end FUs, but there is a high probability that unoccupied FUs are available in adjacent sub-cores during such blockages.

In response, we introduce the adjacent computation resource sharing (ACRS) framework to efficiently utilize these unoccupied units among sub-cores. ACRS has two key modules: Shared FU Issue (SF_ISSUE) and Shared FU Write Back (SF_WriteBack). SF_ISSUE monitors the status of operand collectors and functional units, and offloads instructions from blocked sub-cores to unoccupied resources. Meanwhile, SF_WriteBack routes results back to the original sub-core. To minimize wiring overhead, each sub-core is assigned a fixed target core for sharing. We design a series of matching policies and finally filter out the most effective sequential method. Evaluation results show that ACRS improves performance by up to 46.4%, with an average of 14.1% over the traditional partitioned architecture, while reducing energy consumption by 8.3%. Besides, ACRS achieves an additional 12.3% performance improvement compared with the SOTA method.

I. INTRODUCTION

Modern GPUs have grown in complexity to meet the expansion of high-concurrency applications such as artificial intelligence and high-performance computing. To address the increases in power consumption and die size, GPU architectures now typically divide the main processing unit into homogeneous sub-cores [2]–[10], each equipped with its own control logic and computation resources.

However, partitioned architecture can significantly affect execution efficiency at runtime. Barnes et al. [11] revealed that partitioned architectures in streaming multiprocessors (SMs) cause performance drops when computing tasks are distributed among sub-cores with unbalanced workloads, as well as the increased opportunity of register bank conflicts. Our preliminary study further shows that there exists a large optimization space when integrating functional units (FUs), with an ideal

improvement of 28.1%. Based on these works, we aim to explore a practical FU sharing strategy among sub-cores.

According to our observations, it is greatly effective to share FUs during the blockage of operand collectors (OCs). First, OCs often block the pipeline when functional units are unavailable. This is especially true for FUs with limited execution lanes and high occupancy rates. Second, there is a high probability of finding unoccupied FUs in neighboring sub-cores during these blockages. Our analysis shows that in 68.4% of blockage cases, at least one sub-core in the same SM has available FUs of the same type, indicating a significant potential for resource sharing.

However, the resource-sharing approach encounters challenges, particularly with global sharing within an SM. The challenge arises because resource isolation aims to reduce area and energy consumption. Introducing a crossbar-like network for data exchange among the sub-cores could lead to high overhead. Furthermore, integrating additional write-back paths into the register files requires a significant revision of the write-back logic. This integration could increase pipeline blockages, as data from multiple sub-cores would compete for the same write port. Additionally, adding more write ports would further complicate the issue by expanding the register files' size.

To address these challenges, we present adjacent computation resource sharing (ACRS) framework to enhance the utilization of separated functional units and effectively accelerate program execution. This design comprises two hardware components and one pairing strategy: an SF_ISSUE module to reroute blocked OC instructions to unoccupied FUs; an SF_WriteBack module at the FU exit that returns results to the correct register files via the sub-core identifier; and a **Matching Policy** to determine the sharing target sub-core. Specifically, to simplify control and transmission logic, we have shifted from a global sharing model to a more efficient one-to-one sub-core sharing scheme, significantly reducing the crossbar overhead. Moreover, we have implemented a four-entry write buffer at the register file's entrance to temporarily store write data from other sub-cores, thus avoiding the need for extra write ports and mitigating direct pipeline blockages.

The contributions of this paper include,

*Corresponding author.

- A performance analysis of the impact of sharing different resource types and demonstrate that integrating functional units provides the greatest performance boost.
- An ACRS framework that enables adjacent sub-cores to share unoccupied FUs, allocating available resources to busy operand collectors, thus improving load balancing.
- A comprehensive evaluation, which achieves performance improvement of up to 46.4%, with an average of 14.1% over the traditional partitioned architectures, while reducing energy consumption by 8.3%.

II. BACKGROUND

A. Partitioned Architecture of the SM

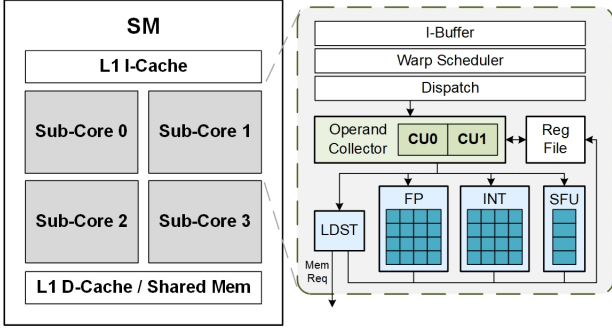


Fig. 1: Baseline SM architecture.

In contemporary GPU architectures, a hierarchical and homogeneous design paradigm is commonly adopted to streamline scheduling and control logic. The design principle is evident at the foundational hardware level, where each basic hardware unit is uniformly divided into four identical segments. For instance, NVIDIA divides a single Streaming Multiprocessor into four sub-cores [2]–[6], while AMD structures each Compute Unit with four SIMD units [7]–[10]. As illustrated in Figure 1, each quadrant of a divided SM possesses its own control and computation resources yet shares a unified L1 data cache and shared memory. Threads are organized into warps that operate synchronously. Once dispatched to a specific sub-core, a warp becomes one of several candidates, with its instructions stored in the I-buffer (instruction buffer) after the fetch and decode stages. Subsequently, an internal warp scheduler orchestrates the distribution of instructions to the operand collector, where they await the retrieval of operands from the register file. Upon readiness of the operands, the instruction is transmitted to either the functional units or the load/store units for further execution.

B. Blocking issues in FUs and OCs

Within each sub-core, multiple functional units process different data types and tasks, including floating point units (FP) for single-precision data, integer units (INT) for integer operations, and special function units (SFU) for complex functions like trigonometry. However, due to hardware limits, a sub-core often cannot process an entire warp simultaneously. For example, each sub-core in the RTX 2060 features 16 FPs and 4 SFUs, yet a warp consists of 32 threads [3]. Consequently, FP instructions require 2 consecutive cycles to be issued to the function units, while SFU instructions take at least 8 cycles.

TABLE I: Baseline simulation parameters.

SM Config	34 SMs, 1600 MHz
Per SM	4 sub-cores, Round-Robin sub-core scheduling
Per sub-core	8 warps, 2 CUs, Greedy-Then-Oldest warp scheduling, 64KB register files, 16 INTs, 16 FPs, 4 SFUs
Cache	128KB L1 I-Cache, 64KB L1 D-Cache/shared memory, 64KB Constant Cache, 4MB L2 Cache

TABLE II: Evaluated applications. The labels **C** and **M** in the Type column stand for compute-intensive and memory-intensive cases, respectively.

Suite	Abbr.	Type	Name
Rodinia [13]	LMD	C	Lava Molecular Dynamics
	KMS	M	K-means Clustering
Dragon-li [1]	JON	C	Dynamic Parallelism Join
Ispass [27]	RAY	C	Ray Tracing
Proxy [15], [16], [28]	XSB	C	XSBBench
	RSB	C	RSBBench
	LLS	M	Livermore Unstructured Lagrangian Explicit Shock Hydrodynamics
Parboil [14]	SPM	M	Sparse-Matrix Dense-Vector Product

This behavior, where a single instruction occupies multiple pipeline stages, can ultimately cause instruction blocking.

The core structure of the sub-core data flow is the operand collector [17], [18], shared by all functional units. The operand collector has multiple collector units (CUs) assigned by the dispatch logic. These units temporarily store instructions fetching operands and are designed to mitigate register file bank conflicts by processing multiple warp instructions concurrently. However, execution around the OCs can stall due to various issues. If no CU is available, the dispatch phase stalls. Additionally, if an instruction occupies multiple execution pipeline stages, especially SFU instructions, subsequent instructions requiring the same unit may stall at the OC exit, delaying execution and reducing efficiency.

III. EXPERIMENTAL ENVIRONMENT

We employ GPGPU-Sim 4.2.0 [12] to simulate a Turing-like architecture, modified from the RTX2060-S configuration provided. Table I presents the baseline parameters. We utilize four sub-cores per SM, mirroring the common 4-core architecture of mainstream GPUs. The methods described in this paper are applicable to architectures with more sub-cores. Additionally, we incorporate two collector units in each sub-core, as this configuration most accurately reflects the actual architecture [11]. This base architecture adeptly models the subdivision of a single SM into sub-cores, facilitating a detailed analysis of the enhancements proposed in this study. Table II presents the representative compute-intensive and memory-intensive applications we select from multiple benchmark suites, including Rodinia [13], Dragon-li [1], Ispass [27], Proxy-apps-doe [15], [16], [28], and Parboil [14]. The selection is based on the applications' characteristics and insights from prior works [21], [29],

trying to ensure a balanced representation of both categories. Previous research [19] indicates that the experimental results tend to converge after approximately 2 billion instructions are executed. We use the largest input sizes for each application and aim to execute each application either until completion or for a maximum of 4 billion instructions.

IV. ANALYSIS

A. Impact of Multiple Partitioned Resources

Each sub-core exclusively possesses four key types of resources in the partitioned architecture: warp scheduler, register files, operand collector, and functional units. Unlike fully connected architectures, these monopolized resources are inaccessible to other sub-cores, even when they are underutilized or left idle. To evaluate the performance impact of individual resources, we share each type of resource within the SM modified from a partitioned structure and then measure the improvements in instructions per cycle (IPC) using the geometric mean, as shown in Figure 2. The following observations are made:

Observation 1: Among the four shared resources, the sharing of functional units and operand collectors both significantly boost performance, with functional units showing the most notable benefit in compute-intensive cases, resulting in an average IPC improvement of 28.1%.

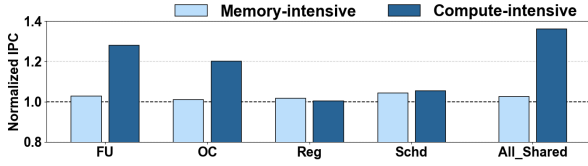


Fig. 2: IPC improvement from shared resources among sub-cores: warp schedulers(Schd), register files(Reg), operand collectors(OC), functional units(FU), and fully connected cores (All_Shared). The results are normalized to the 4-core baseline.

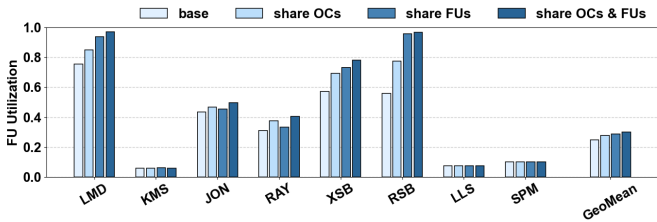


Fig. 3: Comparison of FU utilization for shared structures: shared operand collectors, shared functional units, and their combination, against the 4-core baseline.

Due to the close relationship between operand collectors and functional units, we further investigate whether sharing these two components offers similar benefits. We simultaneously share both operand collectors and functional units, demonstrating their impact on FU utilization as illustrated in Figure 3. FU utilization is defined as the proportion of cycles in which FU is active out of the total number of running cycles. From this figure, we make two main observations:

Observation 2: In memory-intensive cases (KMS, LLS, SPM), FU utilization remains below 20%, and sharing OCs and FUs offers virtually no benefit. This inefficiency stems

from the substantial cache misses in these scenarios, which cause prolonged memory access delays and greatly impede the processing of subsequent instructions.

Observation 3: Simultaneously sharing OCs and FUs does not show a significant difference compared to integrating them separately. However, sharing FUs still outperforms sharing OCs. This is especially true in the cases of LMD, XSB, and RSB, where utilization is the highest among all applications.

Insights: The integration of functional units and operand collectors both significantly influences execution efficiency. Compared to sharing both, focusing solely on FUs achieves a similar impact. Additionally, it avoids the extra overhead associated with sharing OCs.

B. The Sharing Opportunity of Blocking Events

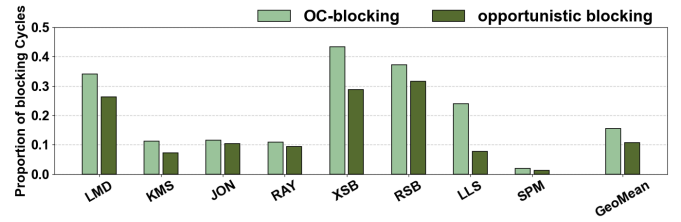


Fig. 4: The proportion of cycles for OC-blocking and opportunistic blocking relative to the total execution cycles.

Pipeline blocking, as described in Section II-B, poses a significant issue in the partitioned architecture. We refer to situations where at least one CU in the OC is blocked as *OC-blocking*. OC-blocking not only diminishes the execution efficiency of the current instruction but also consumes OC resources, potentially hindering the dispatch of subsequent instructions and posing a broader threat to system performance. Figure 4 classifies OC-blocking into two categories: opportunistic and non-opportunistic, and provides statistics on their duration throughout the execution process. *Opportunistic blocking* refers to situations where other sub-cores have similar-type functional units available when the current sub-core experiences OC-blocking. To further understand the sharing potential during OC-blocking, we calculate the conditional probability of opportunistic blocking based on the following equation:

$$P(\text{opportunistic} \mid \text{OC-blocking}) = \frac{F(\text{opportunistic})}{F(\text{OC-blocking})} \quad (1)$$

where $F(*)$ represents the proportion of cycles observed in the total number of execution cycles. We make two key observations from the figure:

Observation 4: OC-blocking is quite prevalent, especially in compute-intensive cases. On average, the execution flow in the programs tested is disrupted by OC-blocking 15.5% of the time, with some tests (LMD, XSB, RSB) experiencing rates as high as approximately 40%.

Observation 5: In all applications tested, the average conditional probability of opportunistic blocking is 68.4%, reaching a peak of 90.5% in the RSB case. This suggests a high probability of reducing the frequency of OC-blocking by utilizing unoccupied units from other sub-cores.

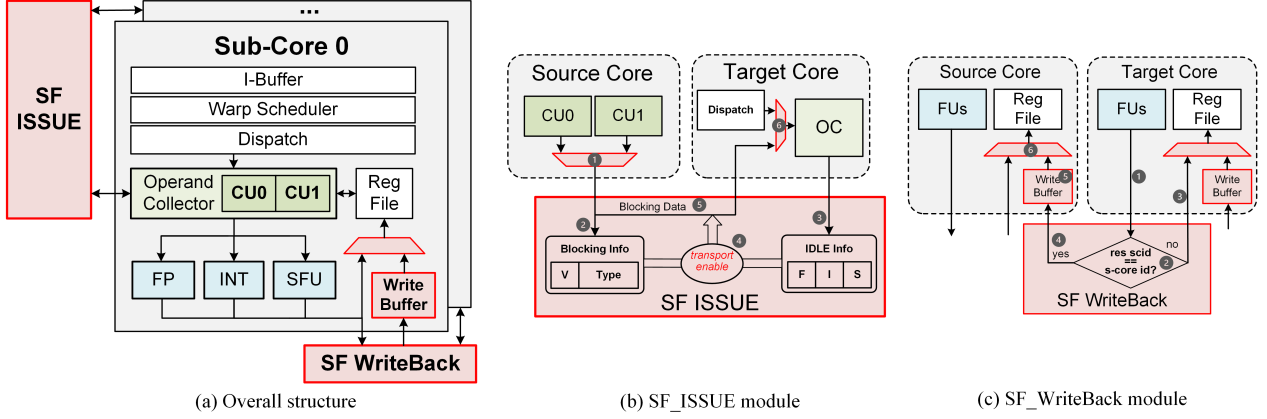


Fig. 5: The proposed ACRS framework.

Insights: OC-blocking is a common issue that impacts execution efficiency. However, there is substantial potential to mitigate these blockages by sharing computation resources. Specifically, if a sub-core can identify and utilize unoccupied execution units from other cores during OC-blocking episodes, overall performance can be greatly enhanced.

V. DESIGN

A. ACRS Architecture

We propose the implementation of ACRS (adjacent computation resource sharing), which is designed to share computation resources among specific sub-core combinations within the SM, alleviating pipeline blocking and enhancing execution efficiency. Figure 5(a) illustrates the overall framework of ACRS where the red parts indicate modifications from the baseline architecture. To facilitate resource sharing, multiple additional data flows are introduced among the four sub-cores. The structure is divided into two main components: SF_ISSUE (Shared-FU Issue) and SF_WriteBack (Shared-FU Write Back), which respectively handle cross-sub-core data exchange before and after task execution. Additionally, a small write buffer is added at the write entrance to the register files to temporarily store results from other sub-cores.

Considering routing overhead, not all resources are made accessible to each core. For a specific sub-core, known as the Source Core (S-Core), when a blocking instruction that can be shared is encountered, the designated Target Core (T-Core) evaluates its ability to accept and process this instruction. The sharing relationship is fixed as one-to-one among the four cores to prevent the formation of an expensive crossbar. We explore various matching policies between S-Cores and T-Cores, which clearly affect the ultimate effectiveness of sharing.

SF_ISSUE. Figure 5(b) depicts the issue logic for a set of S-Core and T-Core. When S-Core experiences OC-blocking, a blocking instruction is first selected from two CUs using the loose round-robin strategy (①), and its details are then sent to the SF_ISSUE module for arbitration. The transferred information consists of two components: the Blocking Info (②) and the Blocking Data (⑤). Specifically, the Blocking Info identifies the type and validity of each instruction. Instruction types are classified into F(FP), I(INT), and S(SFU) based on

the back-end functional units. At the T-Core side, the operand collector consistently monitors the idle status of functional units and sends the information to the SF_ISSUE module. The IDLE Info (③) consists of 3 bits corresponding to the three different function units. When no CU is idle, all three bits are 0. If there is an idle CU and the corresponding FU is also available, the respective bit is set to 1. Thus, the high status of IDLE Info bits indicates that the corresponding type of instruction can be promptly sent to the T-Core without further delay.

Once in possession of the Blocking Info from the S-Core and the IDLE Info from the T-Core, SF_ISSUE is able to employ comparison logic (④) to determine their compatibility. The comparison leverages the type from the Blocking Info to examine the three bits of the IDLE Info. If the corresponding bit is set to 1 and the instruction is valid, the Blocking Data will be issued to the T-Core for execution. Data from the S-Core is given higher priority (⑥) over data dispatched inside the T-Core and is directly accepted by the operand collector. The design rationale is that the transport enable signal is set high only when the execution flow of the S-Core is busy, while the execution flow of the T-Core is comparatively idle, allowing the T-Core to defer processing its own requests.

SF_WriteBack. When the S-Core instruction completes execution in the T-Core, the computation outputs need to be sent back via SF_WriteBack. As illustrated in Figure 5(c), the results (①) from the functional units are directly forwarded to SF_WriteBack. The returned data includes the core ID, which identifies the type of the issuing core. If SF_WriteBack determines the T-Core as the issuing core (②), it confirms local execution and allows direct write-back. Conversely, if the issuing core is the S-Core (④), the results must be routed to the corresponding S-Core. By accurately routing the computation data, the integrity of the SF results is maintained.

The design augments the register write path with a route enabling data return from other cores. To avoid the substantial overhead associated with adding additional ports to the register file, SF_WriteBack incorporates a small WriteBuffer at the entrance to the register file (⑤) to temporarily store computation results from other cores. The local write-back path and the WriteBuffer share a common write port. During arbitration (⑥), the data with the earliest dispatch time is selected for

return to the register file. According to our experiments, a write buffer with a capacity of four entries is sufficient for collecting externally returned data.

Hardware Overheads. Compared to a crossbar used for global sharing, the structure proposed in this paper achieves a significant reduction in layout and routing overhead. The area complexity of a crossbar scales as $O((pw)^2)$ [42], while its scheduler incurs an additional overhead of $O(p^4)$ [43], where p denotes the number of ports and w represents the width of each port. By employing a fixed matching policy, the proposed approach reduces the overhead to $O(pw)$, effectively avoiding the exponential growth in area complexity. Apart from the wiring logic, the primary hardware overhead is the write buffer at the register file interface. Each buffer consists of four entries, with each entry capable of storing a 32-bit result for one warp, totaling 128 bytes. For 64-bit operations, each instruction occupies two buffer entries. Consequently, the total size of the write buffer for each sub-core is 512 bytes, which is negligible compared to the 64KB register files.

B. Matching Policies

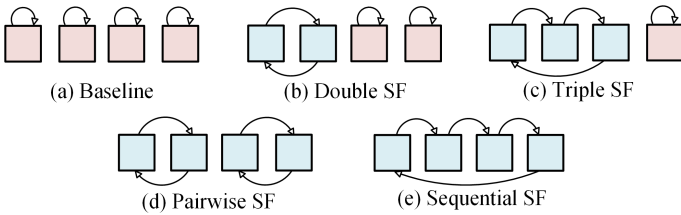


Fig. 6: Matching policies among four sub-cores. Each connection line represents a sharing relationship (from S-Core to T-Core). Blue squares are sub-cores participating in sharing, while red squares are not. We focus on the latter four policies.

Based on the structure proposed in section V-A, each sub-core is required to serve as both an S-Core and a T-Core. With four sub-cores in the SM, four sets of S-Cores and T-Cores are needed. Treating all four sub-cores as identical, we enumerate all possible matching policies for the shared functional units (SF) architecture based on topological principles, as illustrated in Figure 6. In the diagram, each square symbolizes a sub-core, and the connecting lines demonstrate the pairings from S-Cores to T-Cores. Figure 6(a) illustrates the baseline scenario with no sharing, where each core serves as its own S-Core and T-Core. Figure 6(b) shows the *Double SF* configuration, in which two cores share mutually, each acting as the T-Core for the other, while the other two cores remain unshared. Figure 6(c) depicts *Triple SF*, which divides the sub-cores into groups of three and one, with the trio employing a sequential sharing method and the single remaining core not participating in sharing. Figure 6(d), illustrating *Pairwise SF*, divides the four sub-cores into two pairs, with each pair of sub-cores serving reciprocally as T-Cores. Finally, as shown in Figure 6(e), *Sequential SF* policy organizes the sub-cores into a ring, where each subsequent sub-core serves as the T-Core for the previous one.

Notably, we treat all four cores as equivalent, therefore the division of match groups depends only on the number of cores, not on specific cores. For instance, given core IDs from 0 to

3 with parentheses indicating the division into sharing groups, the matching policy for the group (0 1) and (2 3) is considered the same as for the group (0 2) and (1 3), both classified as Pairwise SF. Consequently, the above four matching policies primarily differ in their scope of sharing. Within each group, cores are sequentially assigned a unique T-Core. Compared to the FU sharing method discussed in Section IV-A, referred to as *Ideal SF*, these one-to-one and fixed sharing relationships significantly reduce hardware layout and routing pressure.

VI. EVALUATION

In this section, we evaluate the proposed optimization methods using benchmarks from Table II and simulator configurations from Table I as our baseline.

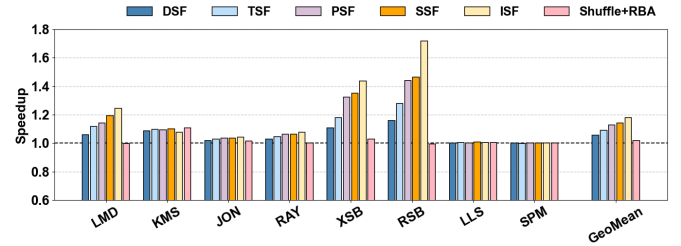


Fig. 7: IPC improvement over the baseline of Shuffle+RBA (SOTA) and our ACRS method with four candidate matching policies: DSF (Double SF), TSF (Triple SF), PSF (Pairwise SF), and SSF (Sequential SF), and the ideal scenario: ISF (Ideal SF).

Performance. Figure 7 illustrates the IPC improvement across all applications under the matching policies described in Section V-B. First, all four matching policies exhibit noticeable improvements in IPC, with larger scopes of sharing resulting in greater enhancements. The sequential matching policy (SSF) shows the largest increase, achieving an average IPC improvement of 14.1%, closely approaching the results of the ISF policy. Second, when all four sub-cores participate in sharing, significant performance improvements are observed across all sharing strategies, with minimal differences among them. The PSF strategy, for instance, achieves an average speedup only 1.3% lower than the best-performing SSF strategy. Third, compute-intensive applications benefit markedly from the shared structure, with the SSF strategy exhibiting up to 46.4% and an average improvement of 19.1%. As for memory-intensive applications, although the enhancements are modest, there are no adverse effects.

Compared with the Shuffle sub-core scheduler and RBA warp scheduler [11] that has a modest 1.8% average performance improvement, our ACRS with SSF matching policy achieves an additional 12.3% improvement, which demonstrates the effectiveness of our approach. The main reason lies in that Shuffle+RBA only focuses on front-end optimization, leaving OC-to-FU stalls unaddressed. The Shuffle scheduler mitigates inter-warp divergence by employing randomized sub-core scheduling patterns, while the RBA scheduler reduces register bank conflicts by sharing queue information of register file banks with the warp scheduler. However, both strategies are unable to resolve the FU separation challenges. In contrast, the ACRS architecture alleviates OC-blocking through FU sharing,

significantly improving pipeline efficiency by addressing back-end bottlenecks.

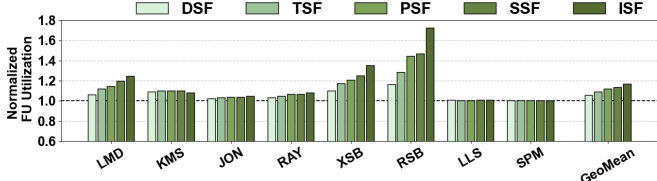


Fig. 8: FU utilization normalized to the baseline.

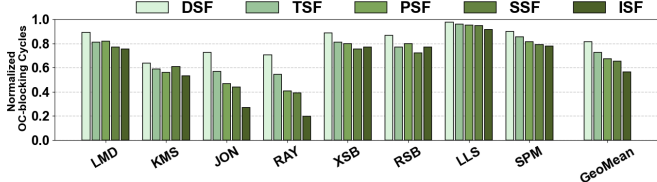


Fig. 9: OC-blocking cycles normalized to the baseline.

We further explore the root causes for the performance improvement and confirm it mainly comes from increased FU utilization and reduction of OC-blocking. Comparing Figure 7, 8, and 9 together, tests showing notable performance increases also exhibit a significant rise in FU utilization and a marked decrease in OC-blocking time. For instance, in the most effective RSB case, the SSF strategy increases FU utilization by 46.5% and reduces the OC-blocking cycles by 27.7%. This observation consistently holds for the top three effective compute-intensive cases (LMD, XSB, RSN). Interestingly, in cases like JON and RAY, although there is a notable reduction in OC-blocking time, the performance improvements are not substantial. This is because OC-blocking is relatively infrequent in these tests and does not represent a bottleneck in the compute pipeline. Therefore, reducing the frequency of OC-blocking has minimal impact on performance in these cases. As for memory-intensive cases (LLS, SPM), their performance is limited by long-latency memory accesses, with memory stalls dominating execution. Consequently, FU utilization remains largely unchanged, and while OC-blocking may be alleviated to some extent, it does not substantially affect overall efficiency due to its minor role in these cases.

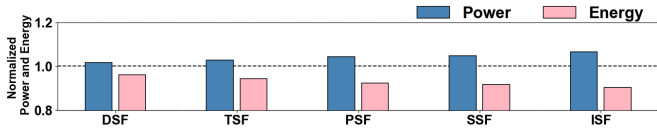


Fig. 10: Normalized power and energy consumption of different matching policies compared with partitioned baseline.

Power. We conduct a power consumption analysis using the AccelWattch platform [20]. The results are normalized and depicted in Figure 10. Compared to the partitioned baseline, all matching policies exhibit an increase in power consumption because they complete the same workload in a shorter period. However, higher power consumption does not necessarily lead to a rise in heat generation. The acceleration of program execution reduces circuit operating time, which in turn decreases static energy consumption and heat produced by control logic

circuits. Notably, under the highly effective SSF strategy, while power consumption increases by 4.7%, energy consumption experiences a more significant reduction, decreasing by 8.3%.

VII. RELATED WORK

Resource Sharing. Various internal GPU sharing strategies at different granularities have been proposed to enhance resource utilization and improve overall performance. [21], [22] focused on optimizing the memory subsystem by enabling the private L1D cache of one SM to be accessible by other SMs. This approach not only reduced data redundancy but also significantly increased memory access efficiency, addressing the bottlenecks commonly associated with memory-bound workloads. [23] introduced BigSM, which meets the computational demands of different kernel types by sharing all resources among adjacent SMs, demonstrating the potential of resource pooling in achieving greater flexibility and scalability in GPU. Tao Zhang et al. [25] shared the pipeline front-end across different SMs, effectively reducing power consumption. Other research [24] targeted sharing at the warp level within an SM by employing a combination of hardware and software techniques to enable register file sharing.

FU Utilization. In addition to sharing methods, previous research has explored different approaches to improve the FU utilization in GPUs. [30]–[34] concentrated on mitigating the impact of branch divergence. By increasing the number of active threads per warp, these approaches effectively reduced the number of idle compute units, ensuring a more balanced workload distribution and enhancing overall throughput in branch-heavy applications. [35]–[37] proposed modifications to warp scheduling strategies aimed at maximizing thread-level parallelism. By carefully prioritizing warps, their method ensured a steadier supply of instructions to the FUs. Meanwhile, [38]–[41] introduced out-of-order execution structures into the SM, utilizing instruction-level parallelism to further reduce pipeline stalls before the execution stage.

Compared to prior works, our approach is the first to explore sharing at the granularity of different SM sub-cores and to primarily focus on functional units as the central sharing objects. Additionally, our approach can complement previous sharing strategies, further improving overall utilization.

VIII. CONCLUSION

In partitioned SM architectures, the separation of sub-core resources can significantly impact execution efficiency. This paper quantifies the performance impact of resource integration, identifying functional units as critical in compute-intensive applications. To address this issue, we propose ACRS, an architecture enabling functional unit sharing between adjacent sub-cores when operand collectors are blocked. We also introduce several sub-core matching policies, with experimental results showing that the sequential policy yields the greatest performance gains.

ACKNOWLEDGMENT

We appreciate the constructive comments and suggestions from all the reviewers. This work is supported by the ICT Innovation Grant E461100.

REFERENCES

- [1] "dragon_li." [Online]. Available: https://github.com/gtcasl/dragon_li
- [2] NVIDIA, "NVIDIA GeForce GTX 750 Ti," 2014. [Online]. Available: <https://international.download.nvidia.com/geforce-com/international/pdfs/GeForce-GTX-750-Ti-Whitepaper.pdf>
- [3] NVIDIA, "NVIDIA Turing GPU Architecture," 2018. [Online]. Available: <https://images.nvidia.com/aem-dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf>
- [4] NVIDIA, "NVIDIA Ampere GA102 GPU Architecture," 2021. [Online]. Available: <https://www.nvidia.com/content/PDF/nvidia-ampere-ga102-gpu-architecture-whitepaper-v2.1.pdf>
- [5] NVIDIA, "NVIDIA Ada GPU Architecture," 2023. [Online]. Available: <https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>
- [6] J. Choquette, O. Giroux, and D. Foley, "Volta: Performance and Programmability," *IEEE Micro*, vol. 38, no. 2, pp. 42–52, Mar. 2018.
- [7] AMD, "Introducing RDNA Architecture," 2019. [Online]. Available: <https://www.amd.com/system/files/documents/rdna-whitepaper.pdf>
- [8] AMD, "Introducing AMD CDNA Architecture," 2020. [Online]. Available: <https://www.amd.com/system/files/documents/amd-cdna-whitepaper.pdf>
- [9] AMD, "AMD CDNA 2 Architecture," 2021. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/instinct-business-docs/white-papers/amd-cdna2-white-paper.pdf>
- [10] AMD, "AMD CDNA 3 Architecture," 2023. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf>
- [11] A. Barnes, F. Shen, and T. G. Rogers, "Mitigating GPU Core Partitioning Performance Effects," in 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA), Montreal, QC, Canada: IEEE, Feb. 2023, pp. 530–542.
- [12] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling," in 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA), Valencia, Spain: IEEE, May 2020, pp. 473–486.
- [13] S. Che et al., "Rodinia: A benchmark suite for heterogeneous computing," in 2009 IEEE International Symposium on Workload Characterization (IISWC), Austin, TX, USA: IEEE, Oct. 2009, pp. 44–54.
- [14] J. A. Stratton et al., "Parboil: A Revised Benchmark Suite for Scientific and Commercial Throughput Computing," 2012.
- [15] J. R. Tramm, A. R. Siegel, T. Z. Islam, and M. Schulz, "Xsbench - The Development and Verification of a Performance Abstraction for Monte Carlo Reactor Analysis," 2014.
- [16] J. R. Tramm, A. R. Siegel, B. Forget, and C. Josey, "Performance Analysis of a Reduced Data Movement Algorithm for Neutron Cross Section Data in Monte Carlo Simulations," in EASC 2014 - Solving Software Challenges for Exascale, Stockholm, 2014.
- [17] S. Liu, J. E. Lindholm, M. Y. Siu, B. W. Coon, and S. F. Oberman, "Operand collector architecture," 7834881, Nov. 16, 2010.
- [18] J. H. Choquette, M. O. Gautho, and J. E. Lindholm, "Methods and apparatus for source operand collector caching," 20130159628, Jun. 20, 2013.
- [19] Z. Lin, M. Mantor, and H. Zhou, "GPU Performance vs. Thread-Level Parallelism: Scalability Analysis and a Novel Way to Improve TLP," *ACM Trans. Archit. Code Optim.*, vol. 15, no. 1, Mar. 2018.
- [20] V. Kandiah et al., "AccelWatch: A Power Modeling Framework for Modern GPUs," in MICRO-54, : 54th Annual IEEE/ACM International Symposium on Microarchitecture, Virtual Event Greece: ACM, Oct. 2021, pp. 738–753.
- [21] H. Falahati et al., "Cross-Core Data Sharing for Energy-Efficient GPUs," *ACM Trans. Archit. Code Optim.*, p. 3653019, Mar. 2024.
- [22] M. A. Ibrahim, O. Kayiran, Y. Eckert, G. H. Loh, and A. Jog, "Analyzing and Leveraging Decoupled L1 Caches in GPUs," in 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA), Seoul, Korea (South): IEEE, Feb. 2021, pp. 467–478.
- [23] A. Dhar and D. Chen, "Efficient GPGPU Computing with Cross-Core Resource Sharing and Core Reconfiguration," in 2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), Napa, CA, USA: IEEE, Apr. 2017, pp. 48–55.
- [24] F. Khorasani, H. Asghari Esfeden, A. Farmahini-Farahani, N. Jayasena, and V. Sarkar, "RegMutex: Inter-Warp GPU Register Time-Sharing," in 2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA), Los Angeles, CA: IEEE, Jun. 2018, pp. 816–828.
- [25] T. Zhang and X. Liang, "Dynamic front-end sharing in graphics processing units," in 2014 IEEE 32nd International Conference on Computer Design (ICCD), Seoul, South Korea: IEEE, Oct. 2014, pp. 286–291.
- [26] A. Jog et al., "Exploiting Core Criticality for Enhanced GPU Performance," in Proceedings of the 2016 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Science, Antibes Juan-les-Pins France: ACM, Jun. 2016, pp. 351–363.
- [27] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, "Analyzing CUDA workloads using a detailed GPU simulator," in 2009 IEEE International Symposium on Performance Analysis of Systems and Software, Boston, MA, USA: IEEE, Apr. 2009, pp. 163–174.
- [28] I. Karlin, "LULESH Programming Model and Performance Ports Overview," LLNL-TR-608824, 1059462, Dec. 2012.
- [29] A. Sethia, D. A. Jamshidi, and S. Mahlke, "Mascar: Speeding up GPU warps by reducing memory pitfalls," in 2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA), Burlingame, CA, USA: IEEE, Feb. 2015, pp. 174–185.
- [30] W. W. L. Fung, I. Sham, G. Yuan, and T. M. Aamodt, "Dynamic Warp Formation and Scheduling for Efficient GPU Control Flow," in Proceedings of the 40th Annual IEEE/ACM International Symposium on Microarchitecture, in MICRO 40. USA: IEEE Computer Society, 2007, pp. 407–420.
- [31] J. Meng, D. Tarjan, and K. Skadron, "Dynamic warp subdivision for integrated branch and memory divergence tolerance," in Proceedings of the 37th annual international symposium on Computer architecture, Saint-Malo France: ACM, Jun. 2010, pp. 235–246.
- [32] W. W. L. Fung and T. M. Aamodt, "Thread block compaction for efficient SIMT control flow," in 2011 IEEE 17th International Symposium on High Performance Computer Architecture, San Antonio, TX, USA: IEEE, Feb. 2011, pp. 25–36.
- [33] Minsoo Rhu and M. Erez, "The dual-path execution model for efficient GPU control flow," in 2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA), Shenzhen: IEEE, Feb. 2013, pp. 591–602.
- [34] A. ElTantawy, J. W. Ma, M. O'Connor, and T. M. Aamodt, "A scalable multi-path microarchitecture for efficient GPU control flow," in 2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA), Orlando, FL, USA: IEEE, Feb. 2014, pp. 248–259.
- [35] S.-Y. Lee and C.-J. Wu, "CAWS: criticality-aware warp scheduling for GPGPU workloads," in Proceedings of the 23rd international conference on Parallel architectures and compilation, Edmonton AB Canada: ACM, Aug. 2014, pp. 175–186.
- [36] V. Narasiman, M. Shebanow, C. J. Lee, R. Miftakhutdinov, O. Mutlu, and Y. N. Patt, "Improving GPU performance via large warps and two-level warp scheduling," in Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture, Porto Alegre Brazil: ACM, Dec. 2011, pp. 308–317.
- [37] A. Jog et al., "OWL: cooperative thread array aware scheduling techniques for improving GPGPU performance," in Proceedings of the eighteenth international conference on Architectural support for programming languages and operating systems, Houston Texas USA: ACM, Mar. 2013, pp. 395–406.
- [38] K. Kim, S. Lee, M. K. Yoon, G. Koo, W. W. Ro, and M. Annamaram, "Warped-preexecution: A GPU pre-execution approach for improving latency hiding," in 2016 IEEE International Symposium on High Performance Computer Architecture (HPCA), Barcelona: IEEE, Mar. 2016, pp. 163–175.
- [39] X. Gong, X. Gong, L. Yu, and D. Kaeli, "HAWs: Accelerating GPU Wavefront Execution through Selective Out-of-order Execution," *ACM Trans. Archit. Code Optim.*, vol. 16, no. 2, pp. 1–22, Jun. 2019.
- [40] K. Iliakis, S. Xydis, and D. Soudris, "Repurposing GPU Microarchitectures with Light-Weight Out-Of-Order Execution," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 2, pp. 388–402, Feb. 2022.
- [41] R. Huerta, J.-M. Arnau, and A. Gonzalez, "Simple Out of Order Core for GPGPUs," in Proceedings of the 15th Workshop on General Purpose Processing Using GPU, Montreal Canada: ACM, Feb. 2023, pp. 21–26.
- [42] N. E. Jerger, T. Krishna, and L.-S. Peh, On-Chip Networks. in Synthesis Lectures on Computer Architecture. Cham: Springer International Publishing, 2017.
- [43] G. Passas, M. Katevenis, and D. Pnevmatikatos, "VLSI micro-architectures for high-radix crossbar schedulers," in Proceedings of the Fifth ACM/IEEE International Symposium on Networks-on-Chip, Pittsburgh Pennsylvania: ACM, May 2011, pp. 217–224.